METRO: MAP-Enhanced Twin Rollouts for Reinforcement Learning-Based Coverage and Capacity Optimization

Jindan Li
jl4767@cornell.edu
Cornell University
New York, USA

Samuel Albert
s.albert@samsung.com
Samsung Research America
Berkeley Heights, USA

Yan Xin
yan.xin@samsung.com
Samsung Research America
Berkeley Heights, USA

Jianzhong Zhang
jianzhong.z@samsung.com
Samsung Research America
Plano, USA

Tianyi Chen
tianyi.chen@cornell.edu
Cornell University
New York, USA

Abstract

Few-shot coverage and capacity optimization (CCO) under a small adjustment budget is challenging: model-based methods suffer from simulator mismatch, while feedback-based methods are sample-starved. We address this challenge with a rollout-based policy-improvement framework. Rather than deploying the trained policy's highest-probability action directly, we use the policy as a proposal mechanism and a maximum a posteriori (MAP)-calibrated digital twin (DT) as the evaluator. The policy samples candidate actions; each is simulated for multiple steps inside the twin, and only the highest-return candidate is deployed. Sparse measurements report signal strength and quality but not user-equipment (UE) locations; we therefore estimate UE locations by MAP inference using DT-predicted measurements as likelihoods and recalibrate the DT to the current network state at each step. Three training-side components strengthen the candidate generation: a Heaviside reward for safety, a recurrent network for partial observability, and domain randomization for simulator mismatch. Ablation studies validate these components. Under a 10-shot emulated-field budget, METRO achieves a lower violation rate than classical simulated annealing and gains comparable to a 50×-budget baseline.

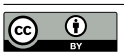
CCS Concepts

• **Networks** → **Network performance evaluation**; • **Computing methodologies** → **Reinforcement learning**.

Keywords

coverage and capacity optimization, reinforcement learning, digital twin, simulator mismatch, AI-native RAN, few-shot deployment

ACM Reference Format:
Jindan Li, Samuel Albert, Yan Xin, Jianzhong Zhang, and Tianyi Chen. 2026. METRO: MAP-Enhanced Twin Rollouts for Reinforcement Learning-Based Coverage and Capacity Optimization. In *The Twenty-seventh International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing (MobiHoc '26), November 23–26, 2026, Tokyo, Japan.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3842721.3856772



This work is licensed under a Creative Commons Attribution 4.0 International License.
MobiHoc '26, Tokyo, Japan
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2975-1/2026/11
https://doi.org/10.1145/3842721.3856772

1 Introduction

Coverage and capacity optimization (CCO) is a multi-objective, multi-cell optimization problem of configuring base-station antenna parameters so that the network provides adequate signal coverage over its service area and sufficient capacity for the traffic offered within that area. Coverage and capacity are inherently coupled: up-tilting an antenna extends reach but spills interference into neighboring cells, depressing their signal-to-interference-plus-noise ratio (SINR). The coupling is also spatial — every parameter change at one cell propagates to its neighbors through interference. Because deploying additional sites or spectrum is far more expensive than re-configuring existing infrastructure, CCO provides operators with a powerful means of extracting value from already deployed networks [1, 3].

Despite its value, CCO is difficult to solve. Two legacy paradigms dominate, each strong where the other is weak. *Model-based* methods optimize inside a ray-tracing (RT) simulator and can extensively explore candidate configurations without impacting the live network. However, their performance is limited by the fidelity of the underlying model: inaccuracies in the map, material properties, and engineering records (e.g., antenna heights and mechanical tilts) can introduce a large *sim-to-real gap*. *Feedback-based* methods drive optimization from live experiments and incur no gap, but each candidate configuration must be physically deployed in a live network. Field exploration is severely sample-starved because operators allow only limited antenna adjustments per optimization campaign [14]. Compounding both paradigms, the network is only partially observable: user-equipment (UE) locations are not reported, and recorded mechanical downtilts may not match the field.

Reinforcement learning (RL) has been proposed as a bridge between these two paradigms. A policy is trained in a simulated network and deployed in a single forward pass: it absorbs the simulator's cheap experience during training, then acts on real measurements at deployment. Previous work has applied deep RL to antenna tuning [8], combined it with a digital twin (DT) [7], and attempted sim-to-real transfer for CCO with few-shot deployment [13]. Under the standard RL deployment paradigm, the trained policy is the decision-maker and its selected action is applied to the network directly. In the few-shot regime, every field action is

costly and cannot be explored freely. Moreover, the policy reflects performance averaged over its training distribution and may not select the best action for its current field observation.

What is needed is an approach that combines the realism of field measurements with the experimental leverage of physics-based modeling. The model reasons about actions that have never been tried, while field feedback corrects the model's latent assumptions about the network. Our method, MAP-Enhanced Twin ROllouts (**METRO**), achieves this by interposing a maximum a posteriori (MAP)-calibrated, RT-based DT between the policy and the field. Rather than acting as the final decision maker, the policy proposes K candidate actions by sampling. Each candidate is evaluated by simulated rollout inside the DT built with Sionna RT [5], and only the best-performing candidate is selected for deployment. Deterministic policy selection is thereby replaced by simulation-evaluated action selection with Proximal Policy Optimization (PPO) [11] serving as the base policy and the DT as the rollout environment. Crucially, the DT is not a fixed simulator. Measurement reports provide sparse received signal strength (RSS) and quality (e.g., SINR) but no UE locations. The DT therefore re-estimates its latent UE locations at each step by MAP inference from the available measurements, using ray-traced predictions as the likelihood model. This feedback continuously re-calibrates the DT to the current network state, allowing candidate actions to be evaluated in the calibrated DT.

Our motivation for rollout is twofold. First, the deterministic model selects an action that was optimized over the average of the environments from training. As the purpose of a DT is to use feedback to better approximate the live environment, rollout over sampled actions gives the opportunity to select an action most suitable for the environment at hand. Second, the highest-probability action may not have the highest return. DT rollout provides estimated Q-values for the candidate actions and reduces this mismatch. Our main contributions are:

- **Deployment-time action evaluation.** The key challenge in few-shot CCO is not only learning a policy, but selecting a good action under a small deployment budget. Instead of directly deploying the policy's best action, we use a calibrated RT-based DT to screen candidates before deployment.
- **A base policy for strong candidates.** Deployment-time evaluation depends on the quality of the proposed candidates. We combine i) a Heaviside reward that penalizes degradation below the initial objective, ii) recurrence to address partial observability, and iii) domain randomization over azimuth offsets to improve robustness to simulator mismatch.
- **Few-shot evaluation.** In a 17-BS Sionna RT emulation with hundreds of UEs, METRO achieves a lower violation rate than CSA with the same 10-shot budget and gains comparable to 50×-budget CSA. Ablations show that the base policy handles partial observations and simulator mismatch, while METRO benefits from policy sampling and DT rollout under limited-BS reports and rollout-DT mismatch.

2 Related Work

Classical and RL-based CCO. Classical CCO spans rule-based controllers that iteratively adjust antenna tilts using real-world call traces [3], coordinate-descent self-optimizing networks [1], and adaptive simulated annealing frameworks from measurement reports [14]; these methods are model-light but either require accurate deployment information or converge slowly under the 10-shot field budget. More recently, deep RL has been applied to 3D multiple-input multiple-output (MIMO) antenna tuning [8], MIMO broadcast beam optimization for self-tuning sectorization [12], and large-scale coverage optimization with a DT and multi-agent PPO [7].

Calibrated DT for sim-to-real and rollout. Our method belongs to the rollout-based policy-improvement family [2]. The DT uses Sionna [5], and prior work shows that such models require field calibration to reduce the sim-to-real gap [4]; we calibrate ours by re-estimating UE locations from field measurement reports via Bayesian MAP inference at every step. Schena et al. [10] also couple a calibrated twin with RL, pairing an adjoint-calibrated twin with model-based optimal control and model-free RL agents. Their DT is primarily used during training, whereas we use the calibrated twin at deployment time to evaluate candidate actions and estimate their returns, enabling rollout-based policy improvement.

3 Problem Formulation and Method

We first formulate few-shot CCO as a finite-horizon POMDP. We then construct a deployment-aware policy that proposes safety-aware and transferable actions under partial observations. Finally, we introduce METRO, which calibrates a digital twin from field measurements and uses it to screen the actions before deployment.

3.1 Problem formulation

We formulate few-shot CCO as a finite-horizon partially observable Markov decision process (POMDP) for sequential BS downtilt adjustment. The scalar objective combines coverage and capacity:

$$\lambda_1 \,\text{Coverage_Metric} + \lambda_2 \,\text{Capacity_Metric}. \tag{1}$$

Here, $\lambda_1, \lambda_2 \geq 0$. Coverage measures whether UEs receive adequate signal strength, for example through RSRP or the fraction of UEs above an RSS threshold. Capacity measures service quality, for example through throughput, RSRQ, or SINR.

In this work, the controllable configuration is the per-BS downtilt vector $\boldsymbol{\theta} = \{\theta^b\}_{b=1}^{\#\text{BS}}$. We instantiate Eq. (1) using the fraction of UEs whose RSS exceeds $\tau_{\text{RSS}} = -105$ dBm and the fraction whose SINR exceeds $\tau_{\text{SINR}} = 3$ dB, assigning equal weight to the two terms:

$$\text{obj}(\boldsymbol{\theta}) = \frac{\#\text{UE}(\text{RSS} \geq \tau_{\text{RSS}}) + \#\text{UE}(\text{SINR} \geq \tau_{\text{SINR}})}{2\#\text{UE}}. \tag{2}$$

Because the downtilts determine the RSS and SINR at each UE, we write $\text{obj}_t = \text{obj}(\boldsymbol{\theta}_t)$ for the objective at decision step t. Here, $\boldsymbol{\theta}_t$ denotes the commanded configuration maintained by the controller and DT; the physical tilt is latent and may differ in the field.

Let $\mathbf{x}_t$ denote the latent physical network state, including unobserved quantities such as the actual UE locations and antenna configuration. In the full-report setting, the agent receives $\mathbf{s}_t$ with three blocks: (i) BS metadata $\mathbf{X}_{\text{bs}} \in \mathbb{R}^{\#\text{BS}\times 5}$, containing each BS's position (x, y), height, transmit power, and carrier frequency; (ii) serving-cell measurements $\mathbf{X}_{\text{ue},t} \in \mathbb{R}^{\#\text{UE}\times 2}$, containing each UE's serving-cell RSS and SINR; and (iii) the BS–UE RSS matrix $\mathbf{R}_t \in \mathbb{R}^{\#\text{UE}\times\#\text{BS}}$. The reduced-report experiments either omit $\mathbf{R}_t$ or report from only the strongest M BSs, as specified in Sec. 4.2 and Sec. 4.4, respectively.

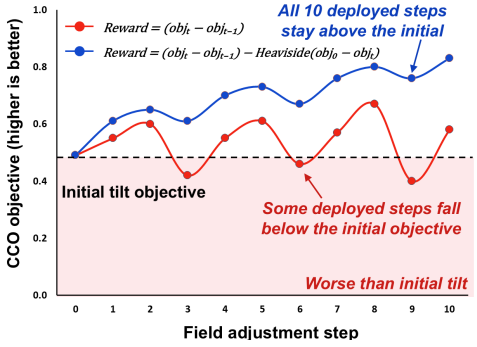


Figure 1: Representative 10-step objective trajectories under the Heaviside and increment-only rewards.

We exclude $\boldsymbol{\theta}_t$ from the policy input. Its effect is already reflected in the measured RSS and SINR, while any difference between the commanded and physical tilts remains latent. At each step, the agent selects a joint discrete downtilt increment $\mathbf{a}_t \in \{-\Delta, \ldots, \Delta\}^{\#\mathrm{BS}}$. The resulting configuration is constrained to remain within ρ degrees of the initial tilt: $\theta^b_{t+1} = \mathrm{clip}\big(\theta^b_t + a^b_t, \theta^b_0 - \rho, \theta^b_0 + \rho\big)$. PPO represents this MultiDiscrete action using one categorical factor per BS, avoiding explicit enumeration of $(2\Delta+1)^{\#\mathrm{BS}}$ joint actions. Each episode has horizon $H = 10$.

3.2 Deployment-aware training

A ten-action deployment must meet three requirements beyond reaching a strong endpoint. First, because every action affects live traffic, temporary degradation should be minimized throughout the trajectory. Second, observations are partial: UE locations are unavailable, and each UE may report only a subset of BS measurements. Third, the training simulator may differ from deployment, creating a *simulator mismatch*. We reserve *rollout-DT mismatch* for errors in METRO's rollout twin, studied in Sec. 4.4. We address these requirements, respectively, with a Heaviside safety penalty, a recurrent policy that aggregates the observation history, and domain randomization over simulator configurations. These training-time modifications strengthen the policy that proposes candidate actions.

Safety-aware reward. We encode pathwise safety by augmenting the stepwise objective improvement with a penalty whenever performance falls below its initial value:

$$r_{t+1} = \underbrace{(\mathrm{obj}_{t+1} - \mathrm{obj}_t)}_{\text{objective improvement}} \underbrace{- C\ \mathrm{Heaviside}(\mathrm{obj}_0 - \mathrm{obj}_{t+1})}_{\text{below-initial penalty}}. \quad (3)$$

Here, $C > 0$ controls the penalty, and $\mathrm{Heaviside}(z) = 1$ for $z > 0$ and 0 otherwise. Thus, the first term rewards progress, while the second distinguishes trajectories that reach similar endpoints but incur different intermediate degradation. The unpenalized, increment-only reward $\hat{r}_{t+1} = \mathrm{obj}_{t+1} - \mathrm{obj}_t$ serves as an ablation. Figure 1 illustrates the resulting behavior; see Sec. 4.1 for quantification.

Recurrence under partial observations. In the reduced-observation setting, the BS–UE RSS matrix $\mathbf{R}_t$ is withheld, leaving only BS metadata and serving-cell RSS and SINR. Because different latent network states can produce similar instantaneous reports, a feed-forward policy cannot resolve this ambiguity from $\mathbf{s}_t$ alone.

We encode each available observation block with a separate two-layer multilayer perceptron (MLP) and concatenate the resulting embeddings into $\phi(\mathbf{s}_t)$. A long short-term memory (LSTM) network then updates $\mathbf{h}_t = \mathrm{LSTM}(\phi(\mathbf{s}_t), \mathbf{h}_{t-1})$, and the policy conditions on $\mathbf{h}_t$ rather than only on the instantaneous embedding. The recurrent state summarizes the observation history and allows the policy to infer task-relevant latent structure from successive sparse reports.

Domain randomization for the simulator mismatch. Domain randomization [15] trains the policy over a distribution of simulator configurations rather than a single nominal scene. For each training scenario, we sample $\delta_{\mathrm{az}} \in \{15^\circ, 30^\circ\}$ and independently draw each BS offset as $\xi^b_{\mathrm{az}} \sim \mathrm{Uniform}\{-\delta_{\mathrm{az}}, +\delta_{\mathrm{az}}\}$. Writing $\mathbf{A}$ for the BS azimuth vector, the perturbed vector is $\mathbf{A}^{(\xi)} = \mathbf{A} - \boldsymbol{\xi}_{\mathrm{az}}$. Offsets are sampled independently across BSs and held fixed throughout an episode. At evaluation, we use the unseen magnitude $\delta_{\mathrm{az}} = 45^\circ$ to test generalization beyond the training distribution. We also randomize UE placement across training scenarios. Together, these variations encourage the policy to base its actions on observed signal patterns rather than memorizing one fixed antenna orientation or UE layout. These modifications address complementary weaknesses of the proposal policy. Even a strengthened policy, however, need not assign its highest probability to the best action for the current field instance. METRO addresses this remaining selection problem by treating the policy as a candidate generator and a field-calibrated twin as the evaluator.

3.3 METRO

METRO alternates two deployment-time operations. At each field step, it first calibrates the twin's latent UE state from the measurement reports collected so far. It then estimates the returns of policy-sampled candidates through simulated continuation in the calibrated twin and deploys the action of the highest-return rollout. We describe the calibration and action-selection stages in turn.

MAP UE-location estimation. The rollout DT requires UE coordinates to evaluate counterfactual downtilts, but field reports do not reveal those coordinates directly. Let $\mathcal{D}_t$ denote all measurement reports collected through field step t. METRO estimates a spatial distribution for each UE from $\mathcal{D}_t$ and updates the estimate after every field action. The RT radio-map solver partitions the scene into grid cells. For UE u, let $p^u_{x,y}$ denote the probability assigned to cell (x, y), and place a Dirichlet prior with concentration $\alpha^u_{x,y}$ on this categorical distribution:

$$\mathrm{Prob}([p^u_{x,y}]) = \frac{\prod_{x,y}(p^u_{x,y})^{\alpha^u_{x,y}-1}}{B([\alpha^u_{x,y}])},\ B([\alpha^u_{x,y}]) = \frac{\prod_{x,y}\Gamma(\alpha^u_{x,y})}{\Gamma(\sum_{x,y}\alpha^u_{x,y})}.$$

Let $\mathcal{B}_{\ell,u}$ be the BSs reported by UE u at field step ℓ. For a scalar RSS report $o_{\ell,b,u}$, define $c^u_{\ell,b,x,y}$ as its likelihood conditional on UE u occupying cell (x, y) under the BS configuration at step ℓ: $c^u_{\ell,b,x,y} = \mathrm{Prob}\big(o_{\ell,b,u} \mid \mathbf{z}^u = (x, y), \boldsymbol{\theta}_\ell\big)$. We model this likelihood as Gaussian and center it at the corresponding DT-predicted measurement. Assuming conditional independence across reports, METRO

solves the following MAP problem separately for each UE:

$$\max_{\{p^u_{x,y}\}} \quad \sum_{\ell=0}^{t} \sum_{b\in\mathcal{B}_{\ell,u}} \log\left(\sum_{x,y} c^u_{\ell,b,x,y} p^u_{x,y}\right) + \sum_{x,y} (\alpha^u_{x,y}-1)\log p^u_{x,y} \quad \text{s.t.} \quad \sum_{x,y} p^u_{x,y} = 1, \qquad p^u_{x,y} \geq 0. \tag{4}$$

The log of a mixture prevents closed-form maximization of Eq. (4). We therefore use Minorization–Maximization [6]. For a fixed UE, we drop the UE superscript and write $\mathcal{B}_\ell = \mathcal{B}_{\ell,u}$ for clarity. At iteration j, define the responsibilities $q^{(j)}_{\ell,b,x,y} = \frac{c_{\ell,b,x,y} p^{(j)}_{x,y}}{\sum_{z,w} c_{\ell,b,z,w} p^{(j)}_{z,w}}$. Jensen's inequality gives a lower bound that is tight at $p^{(j)}$. Holding the responsibilities fixed, the next iterate solves

$$\max_{\{p_{x,y}\}} \quad \sum_{\ell=0}^{t} \sum_{b\in\mathcal{B}_\ell} \sum_{x,y} q^{(j)}_{\ell,b,x,y} \log p_{x,y} + \sum_{x,y} (\alpha_{x,y}-1)\log p_{x,y} \quad \text{s.t.} \quad \sum_{x,y} p_{x,y} = 1, \qquad p_{x,y} \geq 0. \tag{5}$$

The surrogate has a closed-form maximizer. We alternate the responsibility and maximization steps until successive estimates differ by less than a prescribed tolerance. Finally, UE u is assigned to the modal cell $\hat{\mathbf{z}}^u = \arg\max_{(x,y)} p^u_{x,y}$, producing $\hat{\mathbf{Z}}_t = \{\hat{\mathbf{z}}^u\}_{u=1}^{\#\text{UE}}$ for the rollouts at field step t.

Twin rollout for action selection. The MAP step calibrates the twin's latent UE locations but does not itself select an action. Under standard deployment, a feedforward policy would directly apply $\mathbf{a}^{\text{det}}_t = \arg\max_{\mathbf{a}} \pi(\mathbf{a} \mid \mathbf{s}_t)$. For the recurrent policy, the hidden state $\mathbf{h}_t$ defined above already incorporates the current observation. We therefore write $\mathcal{I}_t = \mathbf{h}_t$ and use $\pi(\cdot \mid \mathcal{I}_t)$.

METRO instead screens actions through simulation (Algorithm 1). The policy samples K candidate first actions from $\pi(\cdot \mid \mathcal{I}_t)$. Each candidate is evaluated in a separate DT branch initialized from the same complete current state, including the commanded downtilt, MAP-estimated UE locations, current and initial objectives, and recurrent state. Field reports therefore calibrate the state on which actions are evaluated, while the physics-based DT provides counterfactual evaluations of actions that have not been tried in the field. Figure 2 summarizes this closed-loop interaction.

For candidate k, the DT first applies $\mathbf{a}^{(k)}_t$ and produces reward $r^{(k)}_{t,0}$ and simulated observation $\hat{\mathbf{s}}^{(k)}_{t,1}$. The rollout then follows greedy policy actions for depths $h = 1, \ldots, H-t-1$. Each recurrent branch maintains and updates its own copy of the hidden state from its simulated observation sequence. The candidate score is $\hat{Q}^\pi(\mathcal{I}_t, \mathbf{a}^{(k)}_t) = \sum_{h=0}^{H-t-1} \gamma^h r^{(k)}_{t,h}$. After all candidates are evaluated, METRO selects $\mathbf{a}^*_t = \arg\max_k \hat{Q}^\pi(\mathcal{I}_t, \mathbf{a}^{(k)}_t)$. Only this first action is deployed; all simulated continuation actions are discarded. Candidate screening therefore consumes simulation but does not increase the H-step field-interaction budget.

METRO thus uses each field report to recalibrate the twin and condition the next candidate generation round, closing the loop between measurement, simulation, and control. Section 4 evaluates the training and deployment stages separately.

Algorithm 1 METRO for closed-loop control

Require: Trained policy π; rollout DT; initial reports $\mathcal{D}_0$; initial observation $\mathbf{s}_0$; initial state $\mathbf{h}_{-1}$; candidates K; horizon H

for $t = 0, \ldots, H-1$ **do**
  $\mathbf{h}_t \leftarrow \text{LSTM}(\phi(\mathbf{s}_t), \mathbf{h}_{t-1})$; $\hat{\mathbf{Z}}_t \leftarrow \text{MAP}(\mathcal{D}_t)$
  for $k = 1, \ldots, K$ **do**
    reset DT branch k to the complete current state
    $\hat{\mathbf{h}}^{(k)}_{t,0} \leftarrow \mathbf{h}_t$; $\mathbf{a}^{(k)}_t \sim \pi(\cdot \mid \mathbf{h}_t)$; $r^{(k)}_{t,0}, \hat{\mathbf{s}}^{(k)}_{t,1} \leftarrow \text{env.step}\left(\mathbf{a}^{(k)}_t; \hat{\mathbf{Z}}_t\right)$
    for $h = 1, \ldots, H-t-1$ **do**
      $\hat{\mathbf{h}}^{(k)}_{t,h} \leftarrow \text{LSTM}\left(\phi(\hat{\mathbf{s}}^{(k)}_{t,h}), \hat{\mathbf{h}}^{(k)}_{t,h-1}\right)$; $\mathbf{a}^{(k)}_{t,h} \leftarrow \arg\max_{\mathbf{a}} \pi(\mathbf{a} \mid \hat{\mathbf{h}}^{(k)}_{t,h})$
      $r^{(k)}_{t,h}, \hat{\mathbf{s}}^{(k)}_{t,h+1} \leftarrow \text{env.step}\left(\mathbf{a}^{(k)}_{t,h}; \hat{\mathbf{Z}}_t\right)$
    end for
    $\hat{Q}^\pi\left(\mathcal{I}_t, \mathbf{a}^{(k)}_t\right) \leftarrow \sum_{h=0}^{H-t-1} \gamma^h r^{(k)}_{t,h}$
  end for
  $\mathbf{a}^*_t \leftarrow \arg\max_k \hat{Q}^\pi\left(\mathcal{I}_t, \mathbf{a}^{(k)}_t\right)$
  deploy $\mathbf{a}^*_t$; receive $\mathbf{s}_{t+1}$ and append reports to $\mathcal{D}_{t+1}$
end for

4 Evaluation

All experiments are simulation based and use Sionna RT in three distinct roles: (i) a training simulator for PPO, (ii) an independently configured environment that emulates the field during ten-shot deployment, and (iii) a rollout DT that evaluates METRO's candidate actions. We refer to the second environment as the *emulated field*; no live-network measurements are used. The evaluation follows the decomposition of Sec. 3. Sections 4.1–4.3 evaluate the three training-side components: the Heaviside reward, recurrence under partial observations, and domain randomization. We then evaluate METRO's deployment-time policy improvement in Sec. 4.4. The domain-randomization experiment varies simulator mismatch, whereas the METRO experiment additionally varies the mismatch between the rollout DT and the emulated field.

We evaluate all methods on a 1 km^2 urban scene in a North American city with 17 BSs and hundreds of UEs whose locations are hidden from the deployed policy and METRO estimator. Each deployment episode contains $H = 10$ emulated-field adjustments. Training uses 400 scenarios across five carrier frequencies. The held-out scenario bank contains 100 scenarios, with 20 per frequency and distinct UE placements. Each experiment draws its applicable cohort from this bank and evaluates all methods in that experiment on the same cohort. PPO uses a learning rate of 2×10^{-5}, clip ratio $\epsilon = 0.4$, discount $\gamma = 0.99$, six parallel environments, and a batch size of 60. We set the per-step downtilt bound to $\Delta = 3^\circ$ and the maximum deviation from the initial tilt to $\rho = 15^\circ$.

The primary feedback-only baseline is classical simulated annealing (CSA), which jointly perturbs BS downtilts under the same constraints. Each BS increment is sampled uniformly from $\{-\Delta, \ldots, \Delta\}$. With δ_{obj} denoting the candidate-minus-current objective, improving configurations are accepted and worse ones with probability $\exp(\delta_{\text{obj}}/T)$. The temperature decays as $T \leftarrow 0.995T$ after each step. CSA-10 uses the same ten-adjustment budget as the learned methods, whereas CSA-500 is a 50×-budget reference.

The PPO actor–critic uses a two-stage architecture. For the full-report observation, the first stage independently encodes $\mathbf{X}_{\text{bs}}$, $\mathbf{X}_{\text{ue},t}$, and $\mathbf{R}_t$ with two-layer Tanh MLPs. Each encoder produces a 128-dimensional embedding, and the three embeddings are concatenated into a 384-dimensional feature vector. A shared two-layer

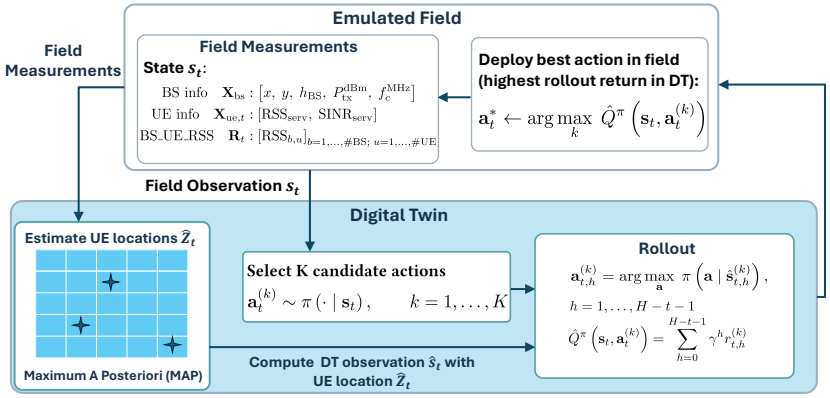


Figure 2: METRO for closed-loop learning-and-control. Top: emulated field, where each action consumes one of ten shots. Bottom: RT-based rollout DT. PPO proposes K candidate joint actions by sampling; each is rolled out for $H - t$ steps in the MAP-calibrated DT to estimate $\hat{Q}^\pi$; the candidate with the largest return is deployed.

trunk then maps this vector from 384 to 256 to 256 dimensions before branching into the action and value heads. Under reduced reporting, the available blocks are encoded in the same modular fashion and concatenated before a dimension-matched shared trunk. The RecurrentPPO variant in Sec. 4.2 inserts an LSTM with 256 hidden neurons between the encoders and the shared trunk.

We report three metrics, each averaged over evaluation scenarios. The *safety-violation rate* is the percentage of scenarios in which the worst-step objective drop from the initial value exceeds 0.5%; lower is better. *Best objective gain* is the relative improvement of the best objective reached during the episode over the initial objective. *Average objective gain* is the relative improvement of the objective averaged over all ten steps over the initial objective.

4.1 Safety via the Heaviside reward

To stress-test trajectory safety, we retain scenarios with initial objectives in $[0.75, 0.95]$, where temporary degradation is especially costly. As Table 1 shows, the Heaviside reward reduces the safety-violation rate from 21% to 7% and outperforms CSA-10 on all three metrics. It nearly matches CSA-500's best gain (6.65% versus 6.80%) while achieving a higher average gain and fewer safety violations.

Table 1: Effect of the Heaviside reward on safe ten-shot CCO.

Method	Violation rate	Best obj. gain	Avg. obj. gain
PPO, Heaviside reward	7%	6.65%	4.31%
PPO, increment-only reward	21%	5.15%	2.96%
CSA-10	67%	3.08%	1.19%
CSA-500	100%	6.80%	1.42%

4.2 Recurrent policy under partial observations

We compare feedforward PPO with RecurrentPPO [9] under full reporting and with $\mathbf{R}_t$ withheld, leaving only BS information and serving-cell RSS and SINR. Training and evaluation use initial-objective ranges $[0.5, 0.95]$ and $[0.65, 0.95]$, respectively. Figure 3 shows training, and Table 2 reports deployment. Removing $\mathbf{R}_t$ raises feedforward PPO's violation rate from 2% to 20% and lowers both gain metrics. With reduced reports, RecurrentPPO improves best gain from 9.39% to 11.26% and lowers the violation rate from 20% to 8%, although average gain decreases from 5.33% to 4.45%. In METRO, recurrence is retained to address partial observability.

Table 2: Effect of recurrence with full/reduced observations.

Method	Violation rate	Best obj. gain	Avg. obj. gain
PPO, full reports	2%	11.34%	8.06%
PPO, reduced obs.	20%	9.39%	5.33%
RecurrentPPO, full reports	8%	11.68%	5.34%
RecurrentPPO, reduced obs.	8%	11.26%	4.45%
CSA-10	46 %	4.92 %	2.37%
CSA-500	93%	11.18%	3.73%

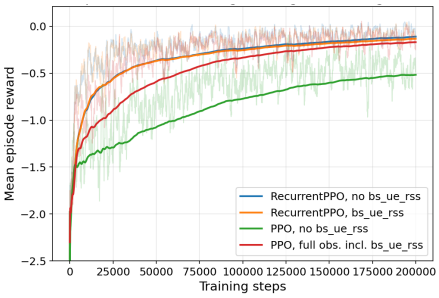


Figure 3: Mean episode reward for RecurrentPPO and PPO, each with and without the BS–UE RSS matrix.

4.3 Domain randomization

Recurrence targets missing measurements; domain randomization instead targets the simulator mismatch. The emulated field applies an independently signed 45° azimuth shift to each BS for the entire episode. We compare nominal training, randomization with 15° and 30° perturbations, and a matched-training reference using the same

45° magnitude as the emulated field. Table 3 shows that domain randomization improves all three metrics: the violation rate falls from 37% to 24%, best gain rises from 6.29% to 8.44%, and average gain rises from 3.74% to 5.24%, therefore, narrows the gap.

Table 3: Domain randomization under unseen azimuth shift.

Method / training config.	Violation rate	Best obj. gain	Avg. obj. gain
No randomization	37%	6.29%	3.74%
Domain randomization	24%	8.44%	5.24%
Matched-training reference	14%	10.51%	7.71%
CSA-10	38 %	5.56 %	3.21%
CSA-500	86 %	12.13%	4.59%

4.4 METRO policy improvement

We evaluate METRO with the domain-randomized RecurrentPPO policy, sampling $K = 100$ candidates at each emulated-field step.

To model limited measurement reporting, each UE supplies RSS from only its $M \in \{3, 6, 17\}$ strongest BSs. These RSS reports are used by the MAP location estimator and populate the available entries of $\mathbf{R}_t$; serving-cell RSS and SINR remain in $\mathbf{X}_{\mathrm{ue},t}$. We consider two rollout-DT conditions. In the *matched-DT* condition, the rollout DT uses the same azimuths and downtilts as the emulated field. In the *mismatched-DT* condition, it has an additional per-BS azimuth offset of ±15° and downtilt offset of ±2° relative to the emulated field. The offsets remain fixed throughout each ten-step episode, representing structural DT error rather than measurement noise.

Under six-BS reporting and a mismatched DT, we compare full METRO with two ablations and two oracle references. The ablations replace policy proposals with random candidates or directly deploy RecurrentPPO's highest-probability action. The references supply exact UE locations with either a mismatched or matched DT. Table 4 shows that learned proposals and rollout both improve action selection. Random proposals degrade every metric, while removing rollout raises the violation rate and reduces gain metrics. MAP and exact locations yield nearly identical gains; the remaining matched-DT-oracle gap is primarily associated with DT mismatch.

Table 4: Ablations: proposals, rollout, and UE localization.

Method	Violation rate	Best obj. gain	Avg. obj. gain
METRO, MAP locations	20.0%	8.19%	5.15%
METRO, random sampling	65.0%	5.04%	2.07%
RecurrentPPO (no rollout)	27.5%	6.94%	4.01%
METRO, exact locations	17.5%	8.04%	5.19%
METRO, matched-DT oracle	12.5%	9.82%	5.96%

Table 5: Limited-BS reports and rollout-DT mismatch.

Reporting BSs	DT condition	Violation rate	Best obj. gain	Avg. obj. gain
3	Matched	22.5%	8.75%	5.40%
3	Mismatched	27.5%	7.56%	4.66%
6	Matched	7.5%	8.96%	5.67%
6	Mismatched	20%	8.19%	5.15%
17	Matched	25%	8.95%	5.31%
17	Mismatched	22.5%	8.59%	5.32%

We next vary the reporting budget and rollout-DT fidelity. Under mismatch, six-BS reporting achieves gains close to 17-BS reporting. The mismatch penalty on gain shrinks with more reports. Together with the preceding ablations, these results show that rollout improves deployment-time action selection, while METRO with MAP-estimated locations retains most of the oracle's objective-gain performance under six-BS reporting and a mismatched DT.

5 Conclusion and Limitations

We have shown that replacing policy deployment with simulation-screened action selection improves few-shot CCO. The trained policy proposes candidates, while the MAP-calibrated DT screens them before deployment. The current evaluation does not quantify UE-localization error or rollout latency. Future work will consider moving UEs, additional layouts, and live-network validation.

References

[1] Sascha Berger, Albrecht J. Fehske, Paolo Zanier, Ingo Viering, and Gerhard P. Fettweis. 2014. Online Antenna Tilt-Based Capacity and Coverage Optimization. *IEEE Wirel. Commun. Lett.* 3, 4 (2014), 437–440. doi:10.1109/LWC.2014.2327228

[2] Dimitri P Bertsekas. 2020. *Rollout, policy iteration, and distributed reinforcement learning.* Athena Scientific, Belmont, MA.

[3] Víctor Buenestado, Matías Toril, Salvador Luna-Ramírez, José María Ruiz-Avilés, and Adriano Mendo. 2017. Self-tuning of Remote Electrical Tilts Based on Call Traces for Coverage and Capacity Optimization in LTE. *IEEE Trans. Veh. Technol.* 66, 5 (2017), 4315–4326. doi:10.1109/TVT.2016.2605380

[4] Ziqi Chen, Yi Ren, Yixuan Huang, Qi Sun, Nan Li, Yuhong Huang, Chih-Lin I, Yifan Li, and Liang Xia. 2026. A Wireless World Model for AI-Native 6G Networks. arXiv:2603.25216 doi:10.48550/ARXIV.2603.25216

[5] Jakob Hoydis, Fayçal Ait Aoudia, Sebastian Cammerer, Merlin Nimier-David, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. 2023. Sionna RT: Differentiable Ray Tracing for Radio Propagation Modeling. In *IEEE Globecom Workshops 2023, December 4-8, 2023.* IEEE, Kuala Lumpur, Malaysia, 317–321. doi:10.1109/GCWKSHPS58843.2023.10465179

[6] Kenneth Lange. 2016. *MM optimization algorithms.* SIAM.

[7] Haoqiang Liu, Tong Li, Fenyu Jiang, Weikang Su, and Zhaocheng Wang. 2024. Coverage Optimization for Large-Scale Mobile Networks With Digital Twin and Multi-Agent Reinforcement Learning. *IEEE Trans. Wirel. Commun.* 23, 12 (2024), 18316–18330. doi:10.1109/TWC.2024.3464639

[8] Yitong Liu, Yubo Shen, Zhe Lyu, Yanping Liang, Wei He, Yuehong Gao, Hongwen Yang, and Li Yu. 2022. Real-Time 3-D MIMO Antenna Tuning With Deep Reinforcement Learning. *IEEE Trans. Cogn. Commun. Netw.* 8, 2 (2022), 1202–1215. doi:10.1109/TCCN.2022.3167549

[9] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8.

[10] Lorenzo Schena, Pedro Marques, Romain Poletti, Samuel Ahizi, Jan Van den Berghe, and Miguel A. Mendez. 2024. Reinforcement Twinning: From digital twins to model-based reinforcement learning. *J. Comput. Sci.* 82 (2024), 102421. doi:10.1016/J.JOCS.2024.102421

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347

[12] Rubayet Shafin, Hao Chen, Young-Han Nam, Sooyoung Hur, Jeongho Park, Jianzhong Zhang, Jeffrey H. Reed, and Lingjia Liu. 2020. Self-Tuning Sectorization: Deep Reinforcement Learning Meets Broadcast Beam Optimization. *IEEE Trans. Wirel. Commun.* 19, 6 (2020), 4038–4053. doi:10.1109/TWC.2020.2979446

[13] Yongxi Tan, Jin Yang, Xin Chen, Qitao Song, Yunjun Chen, Zhangxiang Ye, and Zhenqiang Su. 2018. Sim-To-Real Optimization Of Complex Real World Mobile Network with Imperfect Information via Deep Reinforcement Learning from Self-play. arXiv:1802.06416

[14] Yongxi Tan, Jin Yang, Nandu Gopalakrishnan, Yan Xin, James Mathew, Kamalaharan Dushyanthan, Iyad Alfalujah, and Yanjie Fu. 2016. Autonomous, Closed-Loop and Adaptive Simulated Annealing Based Machine Learning Approach for Intelligent Analytics-Assisted Self-Organizing-Networks (SONs). US Patent App. 14/988,359.

[15] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. 2017. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2017, September 24-28, 2017.* IEEE, Vancouver, BC, Canada, 23–30. doi:10.1109/IROS.2017.8202133